A3-PS024-TC460

Privacy Pass Token Issuance and Anti-Replay Gateway

Team Name: Hacky Potters

AIORI Hackathon 3

Problem Description

Modern web services need to verify that users are authorized to access protected resources without unnecessarily tracking their identities.
Traditional authentication mechanisms may link user identity or browsing activity to access requests. Additionally, multiple legitimate users sharing the same public IP address may be incorrectly treated as a single user.
Reusable tokens can also be exploited through replay attacks, where the same valid token is submitted multiple times.
Key Challenges

- **Privacy: Reduce linkability between token issuance and redemption.**
- **Shared IP Addresses: Support independent access for legitimate clients behind the same proxy or NAT.**
- **Replay Attacks: Prevent tokens from being reused after successful redemption.**
- **Secure Verification: Reject invalid, corrupted, or previously redeemed tokens efficiently.**

Objective: Develop a Privacy Pass-based token issuance and verification gateway using RFC 9578, with an anti-replay mechanism to provide privacy-preserving and secure access to protected resources.

Background

Background

As web services increasingly rely on digital authentication, protecting user privacy while preventing unauthorized access has become an important challenge.

- Privacy Pass (RFC 9578): A privacy-preserving protocol that enables clients to obtain cryptographic tokens and later redeem them without necessarily linking issuance to redemption.
- Token-Based Access: Tokens allow a gateway to verify authorization without requiring users to reveal their identity with every request.
- Anti-Replay Protection: A token that has already been redeemed must not be accepted again. A replay cache such as Redis can help enforce this rule.
- Modern Backend Technologies: Node.js and Express can handle API requests, while Redis supports fast, atomic replay checks.

Project Context: Our project combines Privacy Pass token verification with replay prevention to build a secure, privacy-conscious access gateway.

Literature & RFC Review

1. RFC 9578 — The Privacy Pass Architecture
   - Defines the architecture for privacy-preserving token issuance and redemption.
   - Helps reduce the linkability between token issuance and redemption.
2. RFC 9576 — The Privacy Pass HTTP Authentication Scheme
   - Describes how Privacy Pass tokens are used in HTTP authentication.
   - Provides a standard approach for presenting tokens to a server.
3. Redis — Replay Prevention
   - Provides fast in-memory data storage for tracking redeemed tokens.
   - Supports atomic operations to prevent duplicate redemption.
4. Node.js and Express
   - Provide the backend environment for implementing token issuance APIs, verification endpoints, and gateway logic.

Research Gap / Project Contribution

- Integrates standardized Privacy Pass mechanisms with an anti-replay gateway.
- Focuses on rejecting reused tokens while supporting legitimate clients sharing a public IP address.

Proposed Solution

We propose a Privacy Pass Token Issuance and Anti-Replay Gateway that provides privacy-preserving authorization while preventing token reuse.

Key Components

- **Privacy Pass Issuer: Issues cryptographic tokens using an existing implementation compatible with RFC 9578.**
- **Verification Gateway: Validates submitted tokens before granting access to protected resources.**
- **Anti-Replay Mechanism: Uses Redis to track redeemed tokens and reject duplicate redemption attempts.**
- **Shared-IP Support: Allows legitimate clients behind the same proxy or NAT to use their own independent tokens.**
- **Secure Backend: Uses Node.js and Express to manage API requests, verification workflows, and error handling.**

How It Works:

Issue Token → Client Redeems Token → Cryptographic Verification → Redis Replay Check → Allow or Reject Access

Expected Outcome: Improved privacy, reliable replay prevention, and secure access verification without relying solely on IP addresses to distinguish users.

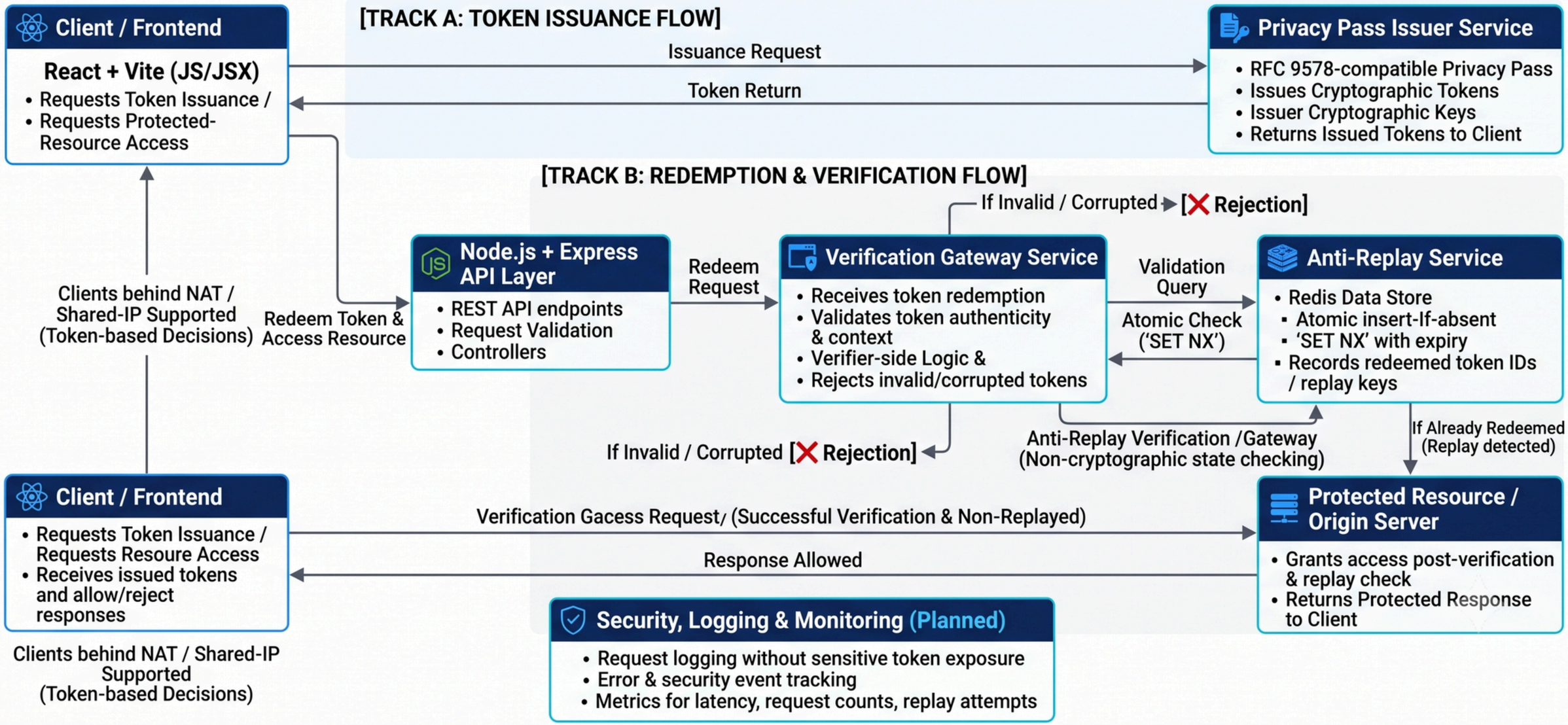
Optimization Proposed

Our project focuses on improving the speed, reliability, and security of the Privacy Pass gateway.

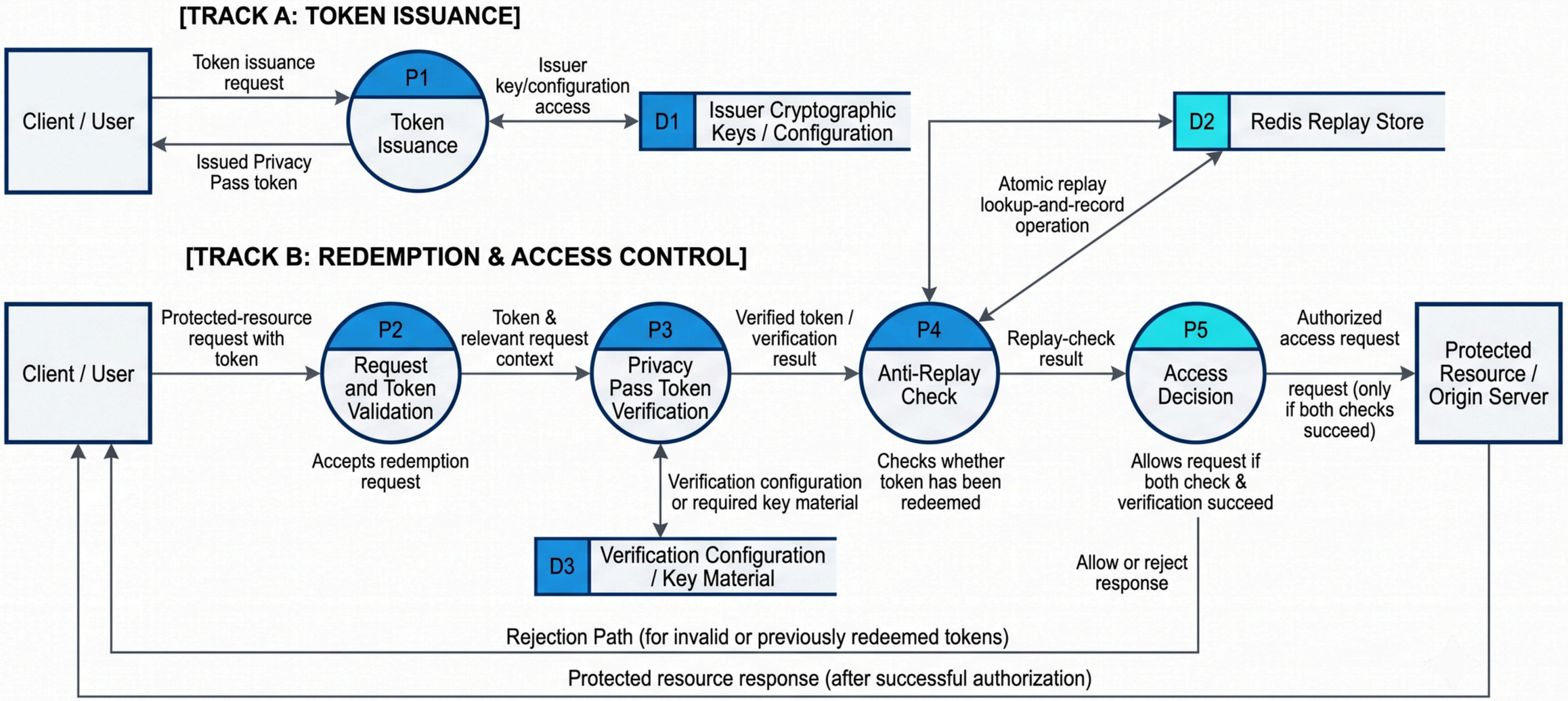
- **Optimized Token Verification: Use an existing RFC 9578-compatible implementation to verify tokens efficiently without developing custom cryptography.**
- **Efficient Replay Detection: Use Redis for fast token lookups and atomic operations to prevent duplicate redemption.**
- **Reduced Response Time: Minimize unnecessary backend processing and validate requests before performing expensive operations.**
- **Concurrent Request Handling: Ensure that simultaneous attempts to redeem the same token cannot both succeed.**
- **Performance Monitoring: Measure verification latency, overall response time, throughput, and replay detection accuracy.**

Expected Outcome: A secure and efficient gateway capable of handling concurrent requests while maintaining privacy and preventing token reuse.

Solution Architecture & Design



Data Flow Diagram



--- TECHNICAL NOTE ---

Security Note: Token verification and replay prevention are separate checks. A valid token must not be accepted again after successful redemption. Replay state is updated atomically (via Redis) to prevent concurrent duplicate acceptance.

Deployment Timeline

Phase 1: Setup & Planning

- **Finalize system architecture, technology stack, and Privacy Pass implementation.**

Phase 2: Backend Development

- **Implement token issuance, verification, and Redis-based anti-replay protection using Node.js and Express.**

Phase 3: Frontend Integration

- **Connect the user interface to backend APIs and display token verification results.**

Phase 4: Testing & Optimization

- **Test valid, invalid, and replayed tokens; evaluate concurrent requests and response latency.**

Phase 5: Deployment & Validation

- **Deploy the application, verify end-to-end functionality, and document results.**

Methodology: Follow an iterative development approach: Build → Integrate → Test → Optimize → Deploy.

Bibliography I

1. RFC 9578: The Privacy Pass Architecture.
2. https://www.rfc-editor.org/rfc/rfc9578
3. RFC 9576: The Privacy Pass HTTP Authentication Scheme.
4. https://www.rfc-editor.org/rfc/rfc9576
5. Node.js Documentation: JavaScript runtime for backend development.
6. https://nodejs.org/docs/latest/api/
7. Express.js Documentation: Framework for building backend APIs.
8. https://expressjs.com/
9. Redis Documentation: In-memory data store for replay prevention.
10. https://redis.io/docs/
11. React Documentation: Library for building the user interface.
12. https://react.dev/